

INF264: INTRODUCTION TO MACHINE LEARNING

PROJECT 2

University of Bergen

2026

Magnus Angel Paulsen & Ingeborg Frigstad Aarbø

Summary

Santa's workshop needs to know which of its programs are safe, and which type of malware each infected program contains. We built three automatic detectors, each trained on about 9,000 example programs with known answers and combined them into a "voting panel" where the majority decides. The detectors look at a small picture of each program. The original data has ten program types, but we removed one, "Installer", because it had only 3 different examples. The detectors therefore work with nine types, the safe programs and eight kinds of malware.

We tested the detectors on 1,631 programs they had never seen. The best single detector (a method called *support vector classifier*) was right for 87.7 out of every 100 programs. The voting panel was right for 87.1 out of 100. It is slightly worse and more complicated, so we recommend the single detector.

Because some program types are much more common than others, we also used a score that gives every type equal weight, macro-F1. On this score, from 0 to 1, our best detector reaches 0.85.

We think machine learning is appropriate for sorting programs by type, but not as the only judge of whether a program is safe.

Our argument in favor of the model is that it is good at catching the eight malware types. For each type the detector found between 90 and 97 out of every 100 programs of that type, which would be slow and hard for people to do by hand.

Our argument against the model is that it is bad at recognizing safe programs. 80 out of 113 safe programs (71%) were wrongly flagged as malware, so Santa's elves would still have to check most of them by hand. Of the 100 programs the detector called "safe", only 33 (33%) were safe.

The most dangerous mistake is a malware program that is classified as safe, a false negative. This happened to 67 out of 1,518 malware programs (4.4%). That is a small share, but one missed infection costs far more than one false alarm. A detector that called everything malware would catch every infection, but it would be useless.

We expect the results to be somewhat worse than 87.7 out of 100, for four reasons. First, the detector cannot recognize "Installer" (class 4), so a real "Installer" program will be misclassified every time. Second, completely new types of malware are not in our data. Third, the mix of programs on Santa's systems may differ from our data. If most programs are safe, for example, the weakness with safe programs will matter much more. Finally, we removed exact copies so that the test would be honest, while real systems contain copies, which changes the results in ways we cannot predict.

We recommend that Blimy use the detector as a first filter that sorts programs by suspected malware type, followed by manual checking. It should not be used alone to declare a program safe, because 67% of the programs it calls "safe" are malware.

Technical report

In this project we build and evaluate a malware classifier for Santa's workshop, whose systems have been infected. The task is to decide, from an image of a program's code, whether the program is safe or belongs to one of several types of malware. The dataset contains 15,563 grey-scale images of 32×32 pixels, each belonging to one of ten classes (one safe class and nine malware classes).

We split the work into the two problems of the assignment. In **Problem 1** we trained three different classifiers, a decision tree (DT), a multilayer perceptron (MLP) and a support vector classifier (SVC), and selected the best one with cross-validated grid search. In **Problem 2** we combined the three tuned classifiers into a hard-voting ensemble classifier and compared it with the best single classifier.

We removed duplicate images and split the data into a training set and a test set. The test set was kept unseen until the final model was chosen, and the models were compared with the macro-F1 score (explained in Section 4). The results are presented in Sections 7 and 8.

The rest of the report is organized as follows. Section 2 describes the data, Section 3 the preprocessing, and Section 4 the performance measure. Sections 5–7 cover Problem 1: the candidate classifiers, the model selection, and the evaluation of the final model. Section 8 covers Problem 2, the ensemble. Section 9 discusses improvements, and Section 10 concludes.

1.1 Division of labor

We worked together throughout the project, with different areas of responsibility. One of us was mainly responsible for the code, and the other for writing the report. All design choices were discussed and decided together. We estimate that the workload was divided roughly equally.

2 Data exploration

Each sample is one program, represented as a flattened 32×32 grey-scale image (a vector of 1,024 values between 0 and 255, where 0 is black and 255 is white). Each pixel is one feature.

The dataset contains 15,563 samples in ten classes (Table 1 and Figure 1) and is clearly imbalanced. The largest class, Crypto (label 6), has 4,613 samples, while the smallest, Installer (label 4), has only 239. A classifier that always answers with the largest class would get a fairly high accuracy without learning anything, so we need another performance measure (Section 4). The only safe class, Good (label 0), has 755 samples, which is small compared with the largest classes. In a malware detector it is more important to separate safe from malicious code than to separate two malware types, so the small size of Good is a concern.

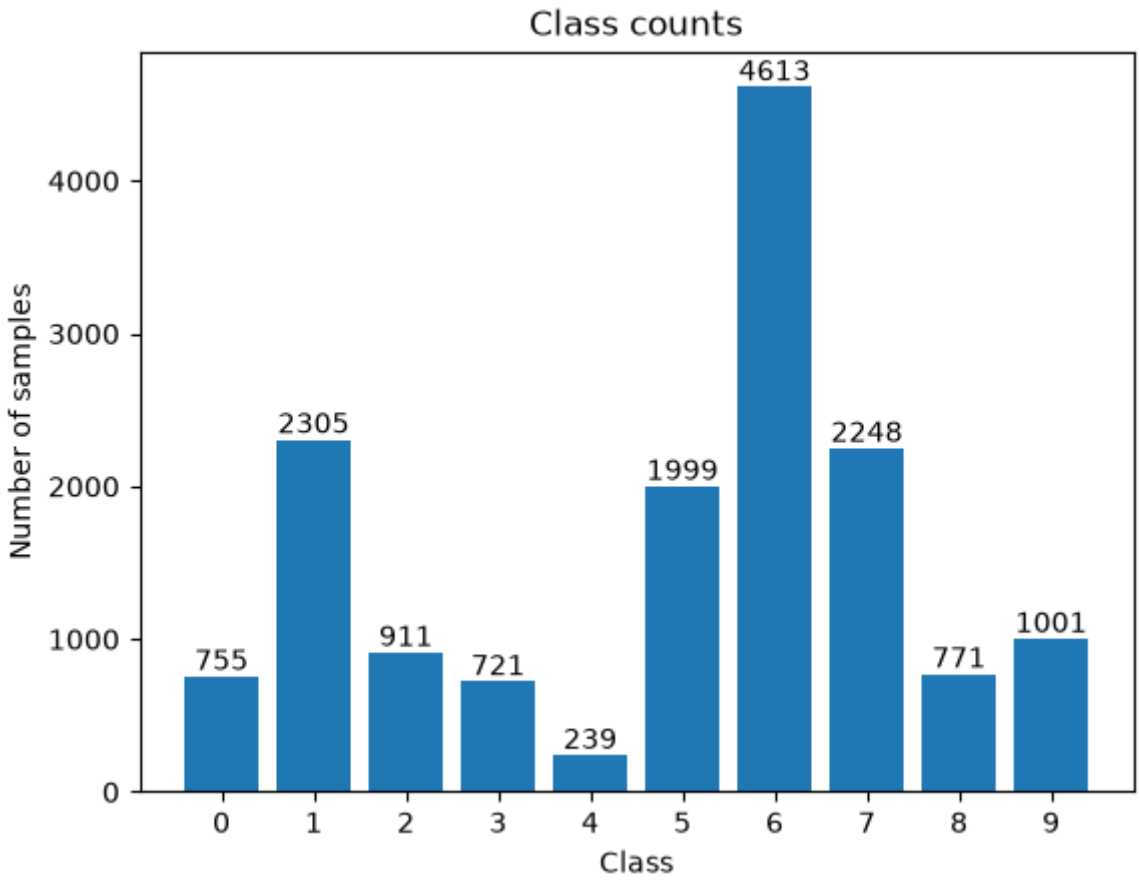


Figure 1: Number of samples per class in the original data.

Using np.unique on the rows of X, we found 10,871 unique images, so 4,692 of the 15,563 samples were exact copies of another sample. The copies are very unevenly spread between the classes (Table 1 and Figure 2). Adware drops from 2,305 to 320 samples and Installer from 239 to only 3, while Good only drops from 755 to 754. Counting only the unique images, Crypto is still the largest class (3,671 samples) and Installer is by far the smallest.

Number	Category name	# Samples before removing duplicates	# Samples after removing duplicates
0	Good	755	754
1	Adware	2305	320
2	Trojan 1	911	830
3	Trojan 2	721	687

4	Installer	239	3
5	Backdoor 1	1999	1646
6	Crypto	4613	3671
7	Backdoor 2	2248	1864
8	Downloader	771	589
9	Heuristic	1001	507

Table 1: Number of samples per class before and after removing duplicates.

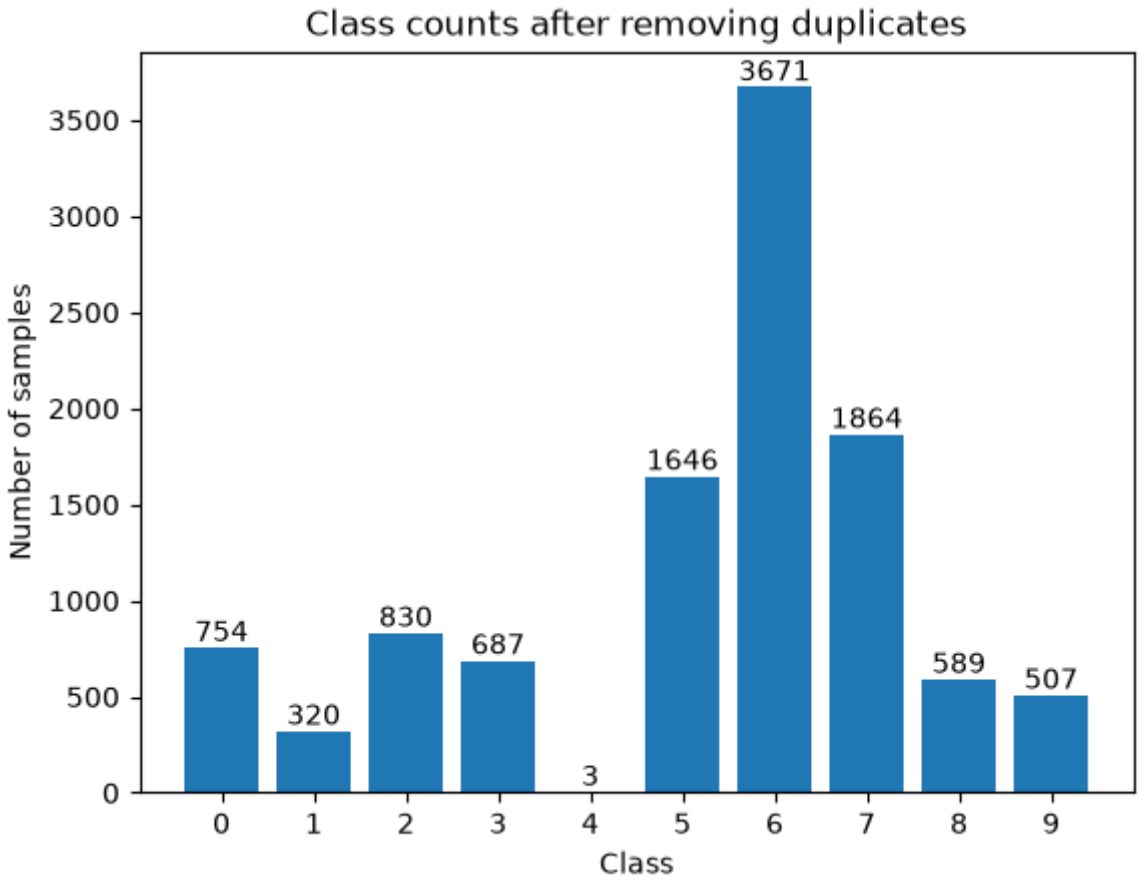


Figure 2: Number of samples per class after removing duplicates.

Figure 3 shows one example image from each class. At first glance the images look like noise, but some classes have clear patterns. Trojan 1 is dark overall, Trojan 2 has a large black area at the bottom, and Adware has a bright area at the bottom. Many of the other classes look quite similar to each other.

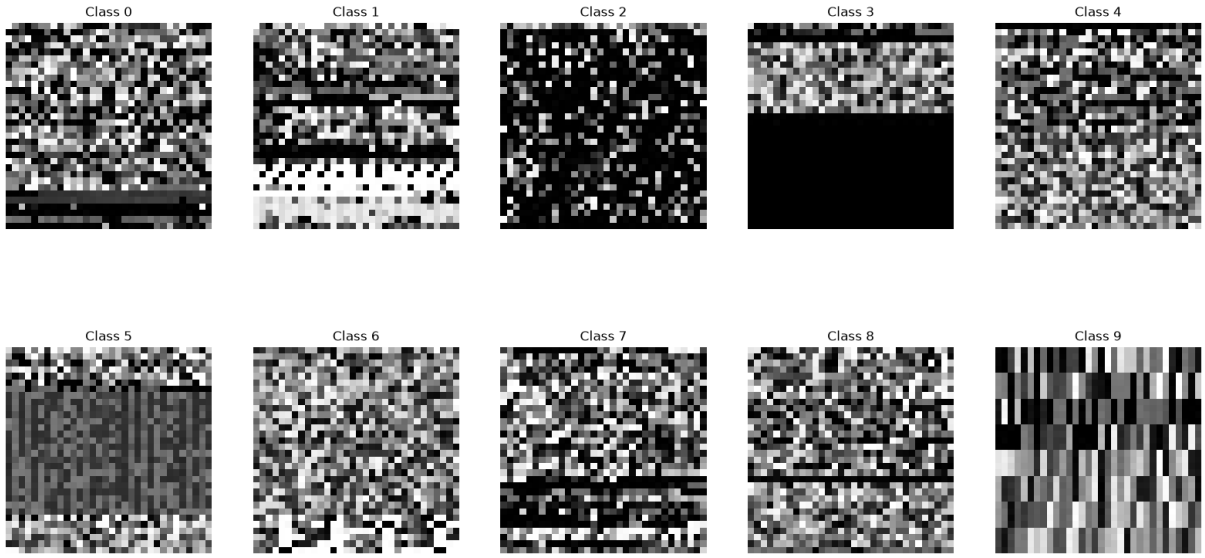


Figure 3: One example image from each class.

The imbalance and the copies affect several of our design choices. The imbalance leads us to use the macro-F1 score (Section 4), a stratified split and class weights. The copies are removed before the data is split, and Installer is removed because it has only 3 unique images (Section 3).

3 Pre-processing

We applied the following steps in this order. First we removed the duplicate images, then the class Installer, and then we split the data into a training set and a test set. Scaling and PCA are done separately for each model, inside a pipeline.

3.1 Remove duplicates

We removed the exact duplicate images and kept the first copy of each (np.unique), which left 10,871 unique images. This was done before the split. If the same image is in both the training set and the test set, the model has already seen the test example during training, and the test score becomes too optimistic. The same problem would appear between the training and validation folds in cross-validation. Identical copies would also get too much weight in the training. Real systems do contain copies, but we want a test that shows how the model performs on programs it has not seen before.

3.2 Removing Installer

After removing the duplicates, Installer has only 3 unique images (from 239 samples). 15% of 3 images is less than one image, so at most one image could end up in the test set, and the rest in the training set. This is too little to both learn the class and evaluate on it. We considered generating synthetic Installer samples, but they would only be variations of three real examples and add little new information. We therefore removed the class. This means that the model cannot detect Installer, and a real Installer program will be placed in one of the nine remaining classes.

3.3 Split in training/test

We split the data into 85% training (9,237 samples) and 15% test (1,631 samples). This keeps as much data as possible for training and model selection, while the test set is still large enough to give a usable estimate. The split is stratified, so the class proportions are the same in both sets. This matters because some classes are small. The random seed is fixed (seed = 67), so the split is reproducible. The test set was not used until the final model had been selected. We do not use a separate validation set, because we use cross-validation on the training set for model selection (Section 6).

3.4 Model-specific scaling

- **Decision tree.** No scaling. A tree only compares one pixel with a threshold, so the scale does not matter.
- **MLP.** MinMaxScaler, which gives values between 0 and 1. Large inputs can explode later in the network, this mitigates it.
- **SVC.** StandardScaler (mean 0 and standard deviation 1 per pixel), followed by PCA, since SVC is also sensitive to the size of the inputs.

We chose the usual/recommended scaler for each model and did not compare the scalers experimentally.

PCA reduces the 1,024 pixels to fewer numbers per image, which keep most of the variation in the data. We use it for the SVC to make the training faster and to reduce noise. The number of components is tuned in Section 6. We did not use PCA for the decision tree, since it splits on single pixels, or for the MLP, which learns its own combinations of pixels. We did not test PCA for those two models.

Scaling and PCA are fitted on data (they compute means, standard deviations and directions). We therefore placed them inside a scikit-learn Pipeline. In cross-validation they are then fitted on the training folds only. If they were fitted on the whole training set first, information from the validation fold would leak into the model.

4 Performance measure

4.1 Macro-averaged F1 score

We use the macro-averaged F1 score to compare the models. It is the scoring metric in the grid search (cross-validation on the training set), and it is also used on the test set for the final models. For a single class, the F1 score is the harmonic mean of precision and recall.

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}, \qquad F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Precision tells us what share of the programs that the classifier assigned to a class really belong to it. Recall tells us what share of the programs that really belong to the class were found. The macro average takes the F1 score of every class k and averages them with equal weight, where K is the number of classes ($K = 9$).

$$\text{macro-F1} = \frac{1}{K}\sum_{k=1}^{K} F\,1_k$$

We chose macro-F1 because all classes should count equally, including the small classes and the safe class Good.

4.2 Why not other measures

We did not use accuracy because the dataset is imbalanced, so accuracy rewards the large classes. A classifier that always answers with the largest class (Crypto) would get a fairly high accuracy without having learned anything. Its macro-F1 would be very low, because it gets an F1 score of zero for every other class.

We did not use weighted F1 either. It averages the class scores weighted by class size, so the large classes dominate. Crypto has about eleven times as many samples as Adware (3,671 against 320 after removing the duplicates), so errors in the small classes would hardly matter.

4.3 Errors with different costs

Macro-F1 treats every misclassification as equally bad, but for Santa the errors are not equally costly. We rank them from worst to least bad as follows.

1. **Malware classified as Good.** The infected program is not checked and can cause damage.
2. **Good classified as malware.** This is a false alarm. Someone has to check the program manually, which costs time and can delay the gift deliveries.
3. **One malware type classified as another malware type.** The program is still flagged, but it may be handled in the wrong way.

We considered a two-step approach (first safe versus malware, then the malware type), but we kept a single classifier and macro-F1 for simplicity. Instead, we report the cost-related quantities separately on the test set (Section 7). Cost-sensitive learning is listed as an improvement in Section 9.

Problem 1

5 Candidate classifiers

We trained three classifiers of different types, which in the course were covered in different lectures, as the assignment requires. Different types also make the ensemble in Problem 2 more useful, since they tend to make different errors. Before running the code we expected the MLP to perform best. The results are presented in Section 6.

5.1 Decision tree (DT)

A decision tree repeatedly splits the training data by asking a yes/no question about one pixel, for example whether the pixel is darker than a threshold. Each final group (leaf) predicts the most common class in the group. We chose it because it is simple and easy to understand, and

a split on one pixel can pick up bright or dark areas in an image. The weakness is that it looks at one pixel at a time, so it is hard for it to combine many weak signals. We tuned criterion, max_depth, min_samples_leaf and ccp_alpha.

5.2 Multilayer perceptron (MLP)

An MLP is a neural network with layers of neurons. It is trained with gradient descent, where the weights are adjusted step by step to reduce the error. We chose it because it can combine many pixels, which suits image data. The weaknesses are that it has many settings and needs scaled input. We tuned the layer sizes, alpha (regularisation) and the learning rate. We also used early stopping, which holds back 10% of the training data and stops training when the validation score stops improving. This made the fits much faster.

5.3 Support vector classifier (SVC)

An SVC finds a boundary between the classes with as large a margin as possible. With an RBF kernel the boundary can be curved, and the model works by comparing how similar the images are. We chose it because it handles many features well and can capture non-linear patterns. We used it after StandardScaler and PCA, and we tuned the number of PCA components, C and gamma.

5.4 Class weights

The decision tree and the SVC were trained with class_weight="balanced". Each class gets the weight

$$w_k = \frac{n_{\text{samples}}}{n_{\text{classes}} \cdot n_k},$$

so errors on samples from small classes are penalised more, and every class has the same total influence on the training. MLPClassifier has no class_weight parameter, so the MLP was trained without it. We therefore expect the MLP to have more difficulty with the small classes. Oversampling the small classes for the MLP is a possible improvement (Section 9).

5.5 Models not used

k-NN works poorly with many dimensions, since distances between images become less informative with 1,024 pixels. The SVC uses a similar idea of similarity between images, so it covers this.

Logistic regression is a linear model, and we expected that a curved boundary would be needed for image data. The SVC with an RBF kernel gives that.

Random forest and boosting are ensembles of trees. They fit better in Problem 2 and are close to the type of the decision tree.

CNN is the standard model for images, but it is outside the course content and needs more computation than a CPU-only setup with the time limit allows.

6 Model selection and hyperparameter tuning

6.1 Procedure

For each of the three model families we ran a grid search with stratified 5-fold cross-validation on the training set (9,237 samples), scoring each candidate with macro-F1. To make the comparison fair, the cross-validation object (StratifiedKFold with shuffling and seed 67), the scorer and the training data are identical for all three searches. After the search, GridSearchCV refits the best candidate on the whole training set (refit=True), so the final model uses all training data. The family with the highest cross-validated macro-F1 is selected. The test set is not used in any of these steps.

6.2 Hyperparameters and grids

Model	**Hyperparameter**	**What it controls**	**Tested values**	**Selected**
DT	criterion	Impurity measure used to choose splits	gini, entropy	gini
DT	max_depth	Longest allowed path from the root to a leaf	16, 24, 32	24
DT	min_samples_leaf	Minimum number of samples in a leaf	1, 2	1
DT	ccp_alpha	Strength of cost-complexity pruning	0, 0.0001, 0.0005	0
MLP	hidden_layer_sizes	Number of hidden layers and neurons in each	(64,), (128,), (128, 64)	(128, 64)
MLP	alpha	L2 penalty on the weights (regularisation)	0.3, 1, 3	1
MLP	learning_rate_init	Step size in gradient descent	0.001, 0.01, 0.1	0.01
MLP	early_stopping	Stop when a validation score (10% of training data) stops improving	True (fixed)	True

SVC	pca__n_components	Number of PCA components kept	35, 50, 75	50
SVC	C	Penalty for misclassified training points (margin versus errors)	0.1, 1, 10	1
SVC	gamma	Reach of each RBF kernel (“bubble” width)	0.002, 0.005, 0.01	0.005

Table 2: Hyperparameters, tested values and the value selected by cross-validation.

Table 2 lists the tested and selected values. We chose coarse grids with a few reasonable steps, because we wanted to keep the run time reasonable (the assignment suggests at most about 30 minutes). The values are rough starting guesses and not derived from theory. A large gamma makes each RBF bubble narrow, so the model follows individual training points closely and risks overfitting. A small gamma makes the bubbles wide and the boundary smooth, which risks underfitting.

6.3 Number of folds

We used 5-fold cross-validation. With 9,237 training samples, a validation fold has about 1,847 samples and each CV model is trained on about 7,390 samples (80% of the training data). More folds would give each CV model more training data, but would also make the run time longer. Five folds kept the total run time within the 30-minute guideline.

6.4 Results and model selection

Model	**Best CV macro-F1**
Decision tree	0.545
MLP	0.767
SVC	0.849

Table 3: Best cross-validated macro-F1 for each model family.

The SVC has the highest cross-validated macro-F1 by a clear margin, so we select it. The selected SVC keeps 50 PCA components and uses C = 1 and gamma = 0.005.

We select on the cross-validated score and not on the test score. If we selected on the test set, it would no longer be an independent estimate, and we would risk picking the model that happens to fit the test samples. The best CV score is also the highest of many candidates (36 for the decision tree, 27 for the MLP and 27 for the SVC), so it is

probably a little too high. This is one more reason why we use the test score, and not the CV score, to estimate the performance of the selected model in Section 7.

6.5 Edge of the grid results and limitation

In our first runs, several selected values lay on the edge of the grid, so we extended or moved the grids around them. With the final grids, all selected values lie inside the tested ranges or at a natural limit.

- SVC: n_components = 50, C = 1 and gamma = 0.005 are all in the middle of the tested values.
- MLP: alpha = 1 and learning_rate_init = 0.01 are both in the middle of the tested values.
- Decision tree: max_depth = 24 is in the middle of the tested values. min_samples_leaf = 1 and ccp_alpha = 0 are the least regularised values possible, so the tree is not limited by the grid. It simply prefers little regularisation.

Since no selected value lies on an open edge of its grid, a value just outside the grid is unlikely to be much better. The grids are still coarse, so better values may exist between the grid points (Section 6.6).

A possible explanation for why the tree prefers little regularisation is that it needs many splits to separate nine classes in 1,024 dimensions. Even so, the tree is far behind the SVC (Table 3).

6.6 Grid search instead of random search

We used grid search and not random search because we had to stay within the time budget. A grid has a fixed and known number of fits, so we could keep the total run time under control. It is also easy to reproduce and interpret. Several of our hyperparameters (C, gamma, alpha) are continuous, so a grid can miss good values between the points, and random search would cover the continuous range better. We accept this risk in return for a predictable run time.

7 Final model and performance evaluation

7.1 The selected model

The selected model is the **support vector classifier with an RBF kernel**, in a pipeline of three steps:

- StandardScaler, which scales each pixel to mean 0 and standard deviation 1 (fitted on the training data).
- PCA with 50 components.
- SVC with RBF kernel, C = 1, gamma = 0.005 and class_weight="balanced".

The RBF kernel measures the similarity between two images as $\exp(-\gamma\|x - x'\|^2)$ in the reduced space, and lets the model separate the classes with a non-linear boundary. C = 1 lies in the middle of the tested values (0.1, 1 and 10). It gives a fairly soft margin that allows some training errors. For nine classes scikit-learn builds 36 binary classifiers (one per pair of classes) and lets them vote.

We selected the SVC because it had the highest cross-validated macro-F1, 0.849 against 0.767 for the MLP and 0.545 for the decision tree (Section 6.4). Possible reasons why the SVC performs best are that PCA removes some pixel noise and that the SVC was trained with class weights. We have not tested these explanations separately.

7.2 Estimated performance on new data

The final model was trained on the whole training set (9,237 samples) and evaluated once on the untouched test set (1,631 samples). Table 4 shows the results.

Model	**CV macro-F1 (training set)**	**Test macro-F1**	**Test accuracy**
SVC (selected)	0.849	0.848	87.7% (1,431 of 1,631)

Table 4: Performance of the selected model. CV = 5-fold cross-validation on the training set; test = held-out test set.

Our estimate of the performance on new data is therefore a macro-F1 of about 0.85 and an accuracy of about 88%. As explained in Section 6.4, we use the test score and not the CV score as our estimate. The test score (0.848) is almost the same as the CV score (0.849). The best CV score was expected to be a little too high, since it is the highest of many candidates, but the test set shows that this bias is small. This agreement is a useful sanity check of our model selection procedure.

The estimate has some caveats. It was computed on deduplicated data with Installer removed, and on a test set with the same class distribution as the training data. Real performance may be lower for new kinds of malware, a different mix of programs, or the Installer class, which the model cannot recognise (Summary).

7.3 Performance per class

Class	Precision	Recall	F1	Support
Good	0.33	0.29	0.31	113

Adware	0.98	0.96	0.97	48
Trojan 1	0.95	0.97	0.96	125
Trojan 2	0.96	0.97	0.97	103
Backdoor 1	0.94	0.90	0.92	247
Crypto	0.97	0.91	0.94	551
Backdoor 2	0.85	0.90	0.87	280
Downloader	0.77	0.94	0.85	88
Heuristic	0.78	0.95	0.86	76

Table 5: Per-class results of the SVC on the test set.

All eight malware classes have F1 between 0.85 and 0.97. The class that stands out is Good, with F1 0.31. Downloader and Heuristic have higher recall than precision, which means that other programs are sometimes assigned to them, mainly Good and Backdoor 1 for Downloader and mainly Crypto for Heuristic (Figure 4).

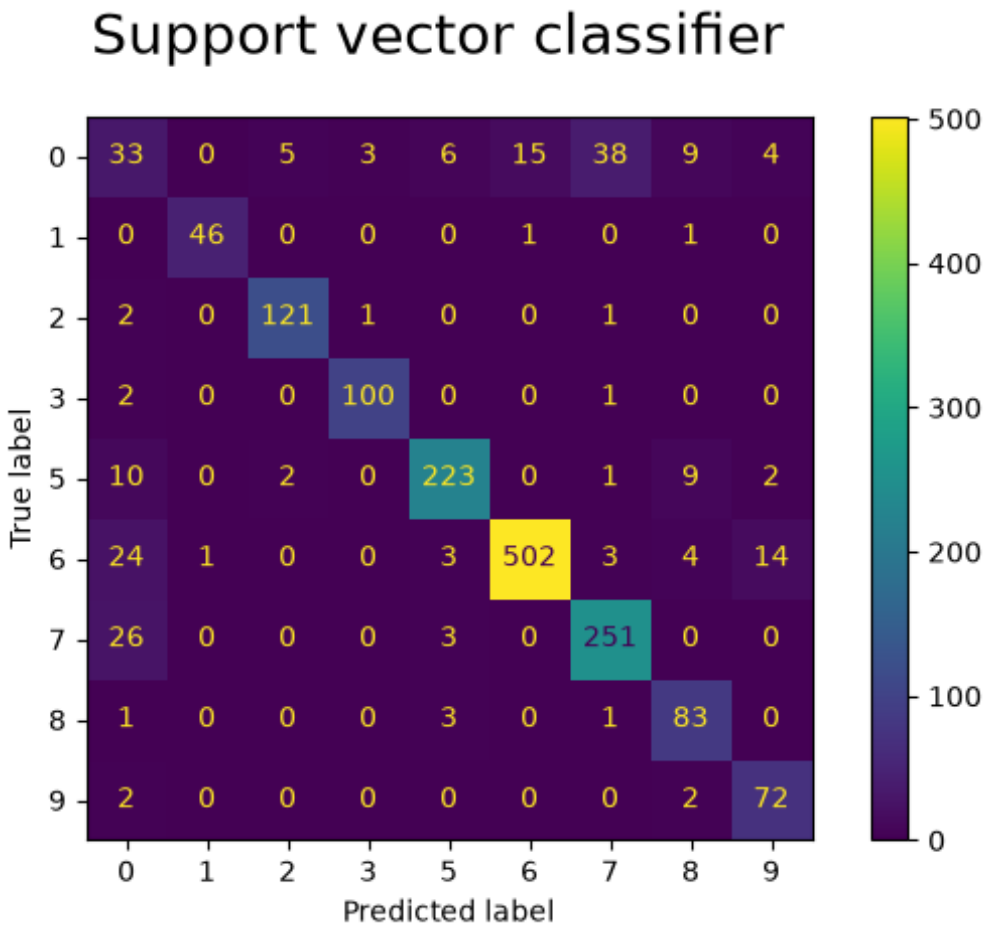


Figure 4: Confusion matrix of the SVC on the test set.

7.4 Low score on Good

Class size alone does not explain the low F1 for Good, since Adware has only 320 samples and reaches an F1 of 0.97. A possible explanation is that Good consists of many different kinds of ordinary programs, so the images in this class vary more than in the malware classes. We have not tested this. Figure 4 shows that the largest confusion is between Good and Backdoor 2.

Table 5 gives every class the same weight, but the most important question for Santa is whether a program is safe (Section 4.3). 80 of the 113 safe programs in the test set (71%) were flagged as malware, so most safe programs would still need a manual check. 67 of the 1,518 malware programs (4.4%) were predicted as Good, which is the most costly type of error. Of the 100 programs the model predicted as Good, only 33 really were Good. A prediction of Good is therefore not reliable, and the model should not be used alone to declare a program safe.

7.5 Measures taken to avoid overfitting

We removed duplicates before the split, so the same image cannot be in both the training set and the test set (Section 3.1). The test set was not used until the final model

had been chosen (Section 6.4). Scaling and PCA are fitted inside the pipeline, so no information leaks from the validation folds (Section 3.4). Each model is regularised through hyperparameters that were tuned with cross-validation, ccp_alpha, min_samples_leaf and max_depth for the tree, alpha and early stopping for the MLP, and C and the number of PCA components for the SVC (Section 6.2).

Problem 2

8 Ensemble

8.1 Implementation

We combined the three tuned models in a scikit-learn VotingClassifier with hard (majority) voting. The members are the best decision tree, the best MLP pipeline and the best SVC pipeline from Problem 1, as the task asks. They come from three different model families, so they can make different kinds of errors. The ensemble is refitted on the full training set, and the SVC inside it is the same model as the SVC we compare with, since it uses the same hyperparameters, seed and training data. Both are evaluated on the same test set.

Each model votes for one class, and the class with the highest total weight wins. We used the weights 1, 1 and 1.1 for the decision tree, the MLP and the SVC. When at least two models agree, their class wins, also when the SVC is the one that disagrees (2.0 against 1.1). Only when all three disagree does the extra 0.1 matter, and then the SVC decides. With a weight of 2 the SVC would tie with the other two models together, and with a weight above 2 the ensemble would be identical to the SVC. A weight of 1.1 therefore keeps majority voting and only uses the best model to break three-way ties.

8.2 Results

Model	Test macro-F1	Test accuracy
SVC	0.848	87.7% (1,431 of 1,631)
Ensemble (hard voting)	0.841	87.1% (1,420 of 1,631)

Table 6: SVC against ensemble on the test set.

The ensemble is not better than the SVC (Table 6). It gets 11 fewer samples right out of 1,631, and its macro-F1 is 0.008 lower. The difference is small, so the ensemble is slightly worse rather than clearly worse. Figure 5 shows that the two models make mostly the same mistakes; the differences are small and spread over several classes.

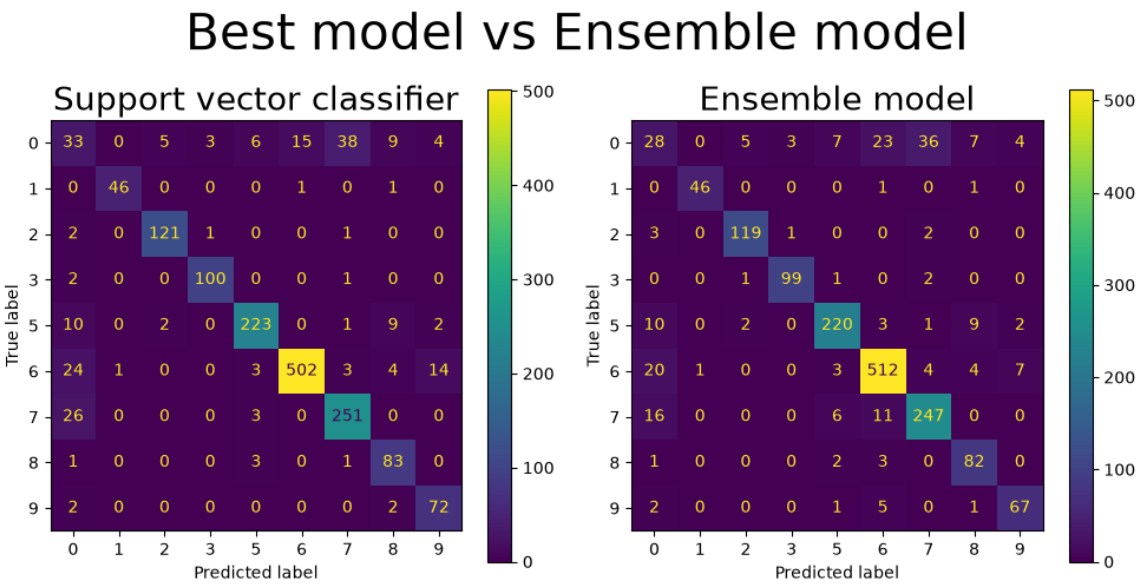


Figure 5: Confusion matrices of the SVC (left) and the ensemble (right) on the test set.

8.3 Why does the ensemble not help?

Majority voting works best when the members are about equally good and make independent errors. Neither holds here.

The models are not equally good. The decision tree and the MLP are clearly weaker than the SVC (Table 3). Since the ensemble only differs from the SVC when these two agree against it, the two weakest models can overrule the best one.

The errors are not independent. The ensemble only differs from the SVC when the decision tree and the MLP agree on another class. This happened on 67 test samples. In 32 of these the SVC was right, in 21 the other two were right, and in 14 none of them were (Figure 6). Whenever the SVC was right, the decision tree and the MLP had made the same mistake, so their errors are clearly related. The ensemble therefore loses 11 samples net.

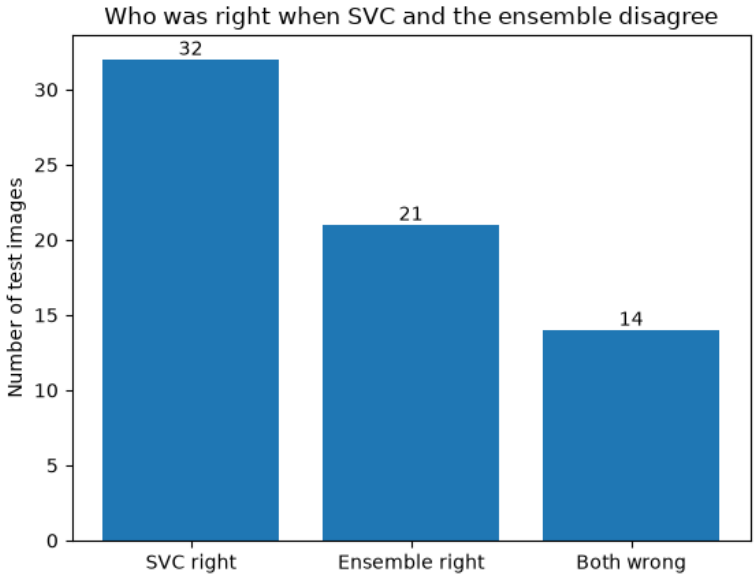


Figure 6: Who was right on the 67 test samples where the SVC and the ensemble disagree.

Figure 7 shows that the ensemble predicts Crypto (class 6) in 43 of the 67 disagreements, most often when the SVC says Backdoor 2 (14 cases) or Heuristic (11 cases). This matches Table 7, where Crypto is the only class the ensemble improves.

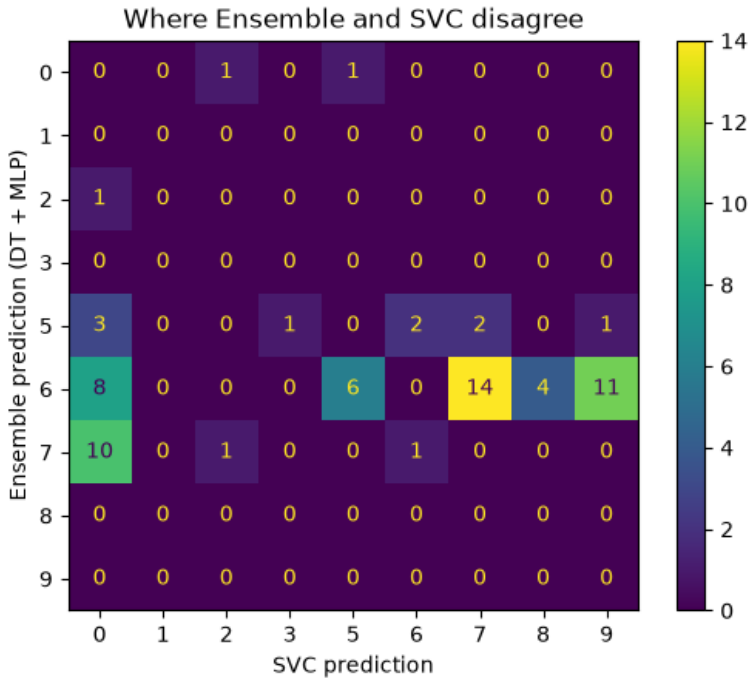


Figure 7: Where the SVC and the ensemble disagree (SVC prediction against ensemble prediction).

Table 7 shows the effect per class. The ensemble gains on Crypto (+10) but loses on Good (−5), Heuristic (−5), Backdoor 2 (−4) and Backdoor 1 (−3). Macro-F1 gives every class the same weight, so losing 5 of 76 Heuristic samples (6.6 percentage points of

recall) counts much more than gaining 10 of 551 Crypto samples (1.8 points). This is why the ensemble's macro-F1 is lower even though it gets more Crypto samples right.

Class	SVC correct	Ensemble correct	Change
Crypto	502	512	+10
Trojan 2	100	99	−1
Good	33	28	−5
Backdoor 1	223	220	−3
Trojan 1	121	119	−2
Heuristic	72	67	−5
Downloader	83	82	−1
Adware	46	46	0
Backdoor 2	251	247	−4

Table 7: Correct predictions per class on the test set (diagonal of the confusion matrices).

Figure 8 shows test images from the three most common disagreements. The SVC is right in most of the Backdoor 2 against Crypto cases, while the ensemble is right in most of the Heuristic against Crypto cases. Neither model is consistently better on these hard images.

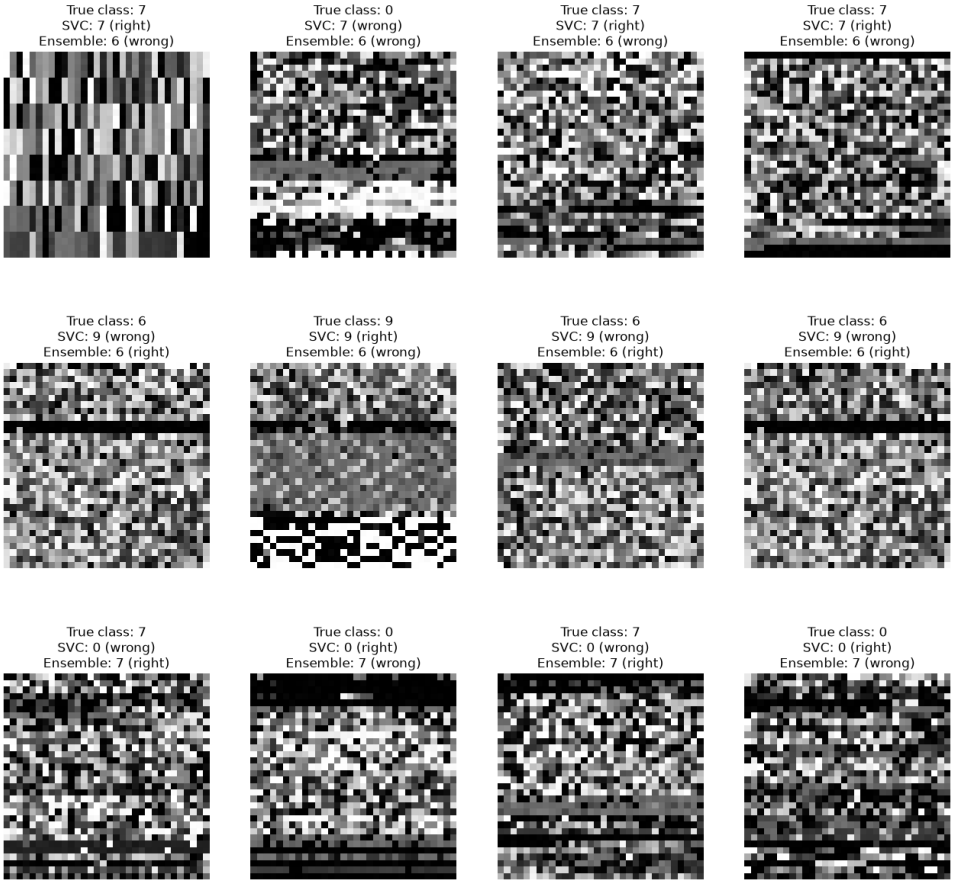


Figure 8: Test images from the three most common disagreements, with which model was right.

8.4 Is the ensemble worth it?

No. It gets slightly fewer samples right than the SVC, we must train and run three models instead of one, and the result is harder to explain. The ensemble does call fewer malware programs safe (52 against 67), which is the most costly error (Section 4.3), but it flags more safe programs as malware (85 of 113 against 80) and is worse on our selection metric. We would deliver the SVC to Santa.

9 Improvements

With more time or computing resources we would do the following. The suggestions follow from the weaknesses we found in the results.

Larger and finer grid searches. Our grids are coarse (Section 6.2), and continuous hyperparameters such as C, gamma and alpha could be tuned more finely, for example with random search or a finer grid around the best values. This needs more run time than we could afford.

Cost-sensitive learning or an adjusted decision threshold. Classifying malware as Good is the most expensive error (Section 4.3). A threshold on the SVC decision_function, larger class weights for the malware classes, or a separate binary safe/malware model first could give fewer missed infections. The price is more false alarms, and 71% of the Good programs are already flagged as malware (Section 7.4), so the change would have to be made carefully.

A better ensemble. The ensemble did not help because the decision tree and the MLP are clearly weaker than the SVC and make similar mistakes (Section 8.3). We would test members that are stronger and more different from each other, and also soft voting and weights based on the cross-validated score.

More real data for Good and Installer. Good is the weakest class (F1 0.31), and Installer has only 3 distinct examples. More varied real examples would help most. Oversampling the small classes could be tried for the MLP, which cannot use class_weight, but only inside the training folds so that no information leaks into the validation data.

Other model types and larger images. A convolutional neural network (CNN) could help if the images contain local patterns, but we do not know whether they do. It would also be an advantage to have larger images than 32 × 32 pixels, since small images may lose information about the programs.

10 Conclusion

We built three classifiers for detecting malware in images of code: a decision tree, an MLP and an SVC. We selected the best one with grid search and cross-validation, and combined the three in a voting ensemble. The data were imbalanced and contained about 30% exact duplicates (15,563 samples before and 10,871 after), which we removed before splitting to avoid data leakage. We also removed the class Installer, which had only 3 distinct images.

The best model was an SVC with an RBF kernel, scaling and PCA. It had a cross-validated macro-F1 of 0.849 and a test macro-F1 of 0.848 (accuracy 87.7%). The hard-voting ensemble was slightly worse (test macro-F1 0.841) and is not worth its extra cost. We therefore recommend the SVC, used as a first filter that sorts programs by suspected type and followed by manual checking.

The main limitations are as follows.

1. The class Installer was removed, so the model cannot detect it (Section 3.2).
2. The class Good is poorly separated (F1 0.31), and 71% of the safe programs are flagged as malware, so the model cannot confirm that a program is safe (Section 7.4).
3. The grids are coarse, so better hyperparameters may exist between the grid points (Section 6.5).

10.1 Reproducibility

All randomness is controlled with seed 67. This covers the train/test split (train_test_split), StratifiedKFold, the decision tree, the MLP and the PCA (random_state). The SVC itself is deterministic, since we do not use probability=True, and the VotingClassifier contains no randomness of its own.

We submit the notebook, a README.md that explains how to run it, and a requirements.txt with package versions. The data set is loaded from dataset.npz, which must be placed next to the notebook. The notebook should be run from top to bottom after restarting the kernel. A full run takes about 8 minutes on an AMD CPU with 16 threads. All numbers in this report come from this run.